

Serie III — Proceso de Pruebas de Software

Casos de prueba alineados a los requisitos del proyecto (RF1–RF8)
Sistema de Gestión de Inventario y Precios — Tienda Bugambilias (POS Web)

NOMBRE: SANTOS DAVID TOJ ÁLVAREZ · GUATEMALA, OCTUBRE DE 2026

1. Introducción

Este documento presenta las pruebas de software del proyecto, **trazadas a los requisitos funcionales** definidos en «OBJETIVOS DEL PROYECTO.pdf» (RF1–RF8). Cada requisito funcional cuenta con al menos un caso de prueba ejecutado y aprobado. Las pruebas se ejecutan de forma automatizada con **Jest** contra la API REST real del sistema (PHP 8.3 + MySQL 8), levantada con Docker en `http://localhost:8091`.
Comando: `cd pruebas && npm test`.

Nota de alcance: según el documento de objetivos, el módulo de **Ventas / POS completo está fuera del alcance** y el registro/inicio de sesión de usuarios no es un requisito funcional explícito (solo RF8, «niveles de acceso»). Por ello las pruebas de **Autenticación** y **Ventas** se reportan por separado como *cobertura adicional* (sección 4), no como requisitos.

2. Matriz de trazabilidad: Requisitos ↔ Casos de prueba

RF	Requisito funcional	Caso(s) de prueba	Archivo	Tipo	Resultado
RF1	Crear un nuevo producto (nombre, precio de compra, precio de venta y stock)	CP-PROD-001 · CP-PROD-004 (validación)	productos.test.js	Negra / Blanca	Aprobado
RF2	Actualizar / editar la información de un producto existente	CP-PROD-002	productos.test.js	Negra	Aprobado
RF3	Eliminar un producto del catálogo	CP-PROD-003	productos.test.js	Negra	Aprobado
RF4	Mostrar el stock disponible de cada producto en tiempo real	CP-PROD-001 (el listado muestra el stock actual)	productos.test.js	Negra	Aprobado
RF5	Calcular automáticamente la ganancia individual (venta –	CP-GAN-001 · CP-	ganancias.test.js	Blanca	Aprobado

	compra)	GAN-003			
RF6	Buscar un producto por nombre para consultar precio y stock	CP-PROD-001 (búsqueda por parámetro q: nombre o código)	productos.test.js	Negra	Aprobado
RF7	Listado general de productos con stock, precio y ganancia	CP-GAN-002 · CP-PROD-005	ganancias / productos.test.js	Negra / Gris	Aprobado
RF8	Diferenciar niveles de acceso (administrador vs. empleado)	CP-PROD-005	productos.test.js	Gris	Aprobado

Cobertura de requisitos: 8 de 8 (100 %).

3. Casos de prueba en formato estándar

Caso 1 · RF1 Crear producto Caja negra

productos.test.js — *POST /api/productos*

ID del caso de prueba	CP-PROD-001
Requisito cubierto	RF1 — Crear un nuevo producto (también valida RF4 «mostrar stock» y RF6 «búsqueda» al consultar el listado).
Escenario de prueba	Registro exitoso de un nuevo producto.
Pasos de la prueba	1. Iniciar sesión como administrador. 2. Enviar <i>POST /api/productos</i> con los datos del producto. 3. Buscar el producto por su código/nombre en el listado (<i>GET /api/productos?q=</i>).
Requisitos previos	Sesión activa con rol administrador; el código del producto no existe.
Datos de prueba	código: ARZ-(único) · nombre: Arroz Integral 1 lb · precio de compra: Q6.00 · precio de venta: Q8.50 · stock: 40 · categoría: Granos Básicos.
Resultado esperado	201 Created con el mensaje «Producto creado.»; el producto aparece en el listado con su precio y stock.
Resultado real	201 «Producto creado.»; el listado lo muestra con precio Q8.50 y stock 40.

Estado **Aprobado**

Caso 2 · RF2 Editar producto Caja negra

productos.test.js — *PUT /api/productos/{id}*

ID del caso de prueba CP-PROD-002

Requisito cubierto RF2 — Actualizar / editar un producto existente.

Escenario de prueba Actualización del precio de venta de un producto existente.

Pasos de la prueba

1. Crear un producto con precio de venta Q10.00.
2. Enviar *PUT /api/productos/{id}* con el nuevo precio.
3. Consultar el producto.

Requisitos previos El producto existe en el catálogo; sesión de administrador.

Datos de prueba precio de venta anterior: Q10.00 · precio nuevo: Q12.50.

Resultado esperado 200 OK; al consultar el producto su precio es Q12.50.

Resultado real 200 «Producto actualizado.»; el detalle devuelve precio 12.5.

Estado **Aprobado**

Caso 3 · RF3 Eliminar producto Caja negra

productos.test.js — *DELETE /api/productos/{id}*

ID del caso de prueba CP-PROD-003

Requisito cubierto RF3 — Eliminar un producto del catálogo.

Escenario de prueba Eliminación de un producto del catálogo.

Pasos de la prueba

1. Crear un producto.
2. Enviar *DELETE /api/productos/{id}*.
3. Buscarlo en el listado.

Requisitos previos El producto existe y está activo.

Datos de prueba código: ELI-(único).

Resultado esperado 200 OK; el producto ya no aparece en el listado.

Resultado real 200 «Producto eliminado (baja logica).»; el listado queda vacío para ese código.

Estado **Aprobado**

Caso 4 · RF1 Validación de datos Caja blanca

productos.test.js — *POST /api/productos (ramas de validación)*

ID del caso de prueba CP-PROD-004

Requisito cubierto RF1 — Robustez al crear un producto (datos obligatorios y válidos).

Escenario de prueba Intento de registrar un producto con campos obligatorios vacíos o inválidos.

Pasos de la prueba

1. Identificar las 4 ramas de rechazo de `api_productos_create()`.
2. Enviar un POST por cada rama.
3. Verificar el código, el mensaje y que no se guardó nada.

Requisitos previos Sesión de administrador.

Datos de prueba Rama 1: código/nombre vacío · Rama 2: sin precio · Rama 3: precio -1 o «diez» · Rama 4: código ya registrado.

Resultado esperado Ramas 1–3: 422 con su mensaje de validación, sin guardar. Rama 4: 409 «Ya existe un producto con ese código».

Resultado real 422 / 409 con los mensajes esperados; en ningún caso quedó guardado (7 de 7 verificaciones).

Estado **Aprobado**

Caso 5 · RF5 Cálculo de ganancia Caja blanca

ganancias.test.js — *GET /api/productos/{id}*

ID del caso de prueba CP-GAN-001

Requisito cubierto RF5 — Calcular automáticamente la ganancia individual (venta – compra).

Escenario de prueba Cálculo correcto de la ganancia y el margen de un producto.

Pasos de la prueba

1. Registrar un producto con compra Q10 y venta Q15.
2. Consultar el producto.
3. Comparar con la fórmula: ganancia = venta – compra; margen = ganancia / venta × 100.

Requisitos previos Sesión de administrador.

Datos de prueba precio de compra: Q10.00 · precio de venta: Q15.00.

Resultado esperado ganancia Q5.00 y margen 33.3 %.

Resultado real ganancia 5 y margen 33.3.

Estado **Aprobado**

Caso 6 · RF7 Listado / rentabilidad Caja negra

ganancias.test.js — GET /api/reportes/rentabilidad

ID del caso de prueba CP-GAN-002

Requisito cubierto RF7 — Listado general de productos con stock, precio y ganancia (identificación de los más/menos rentables).

Escenario de prueba El reporte ordena los productos del más al menos rentable.

Pasos de la prueba

1. Registrar dos productos: A (Q10 → Q40) y B (Q10 → Q11).
2. Registrar movimientos de 2 unidades de cada uno.
3. Consultar el reporte de rentabilidad.

Requisitos previos Existen varios productos con ganancias distintas.

Datos de prueba Producto A: gana Q30 por unidad · Producto B: gana Q1 por unidad.

Resultado esperado A genera Q60 y B Q2; A aparece antes que B en el reporte.

Resultado real A: Q60 · B: Q2 · A aparece antes que B.

Estado **Aprobado**

Caso 7 · RF8 RF7 Niveles de acceso Caja gris

productos.test.js — *POST/GET /api/productos* (rol cajero)

ID del caso de prueba CP-PROD-005

Requisito cubierto RF8 — Diferenciar niveles de acceso; además RF7 (el listado filtra la ganancia/costo según el rol).

Escenario de prueba Un cajero intenta crear un producto y consulta el catálogo.

Pasos de la prueba

1. Iniciar sesión como cajero.
2. Enviar `POST /api/productos`.
3. Consultar `GET /api/productos` y revisar los campos de cada producto.

Requisitos previos Sesión activa con rol cajero.

Datos de prueba usuario: cajero@tienda.com · producto: código CAJ-(único).

Resultado esperado 403 al crear; el listado no incluye `precio_compra` ni `ganancia`.

Resultado real 403 «Acceso denegado. Se requiere rol de administrador.»; ningún producto trae `precio_compra` ni `ganancia`.

Estado **Aprobado**

4. Cobertura adicional (fuera de los requisitos funcionales)

Las siguientes pruebas **no corresponden a un requisito funcional del documento de objetivos**, pero están implementadas en el sistema y se ejecutan en la misma corrida. Se incluyen como refuerzo de la calidad.

Adicional Autenticación (Login / Registro) — soporta RF8

Archivo `auth.test.js`. Aunque el inicio de sesión y el registro no son requisitos explícitos, sustentan el control de acceso (RF8). Casos: **CP-LOGIN-001** (login correcto → 200), **CP-LOGIN-002** (contraseña incorrecta → 401), **CP-LOGIN-003** (campos vacíos → 422), **CP-REG-001** (registro exitoso → 201) y **CP-REG-002** (correo duplicado → 409). Resultado: **5 / 5 aprobados**.

Adicional Ventas / POS — fuera del alcance (refuerza RF4)

Archivo `ventas.test.js`. El POS completo está fuera del alcance del proyecto; sin embargo, el registro de ventas permite verificar la **actualización del stock en tiempo real (RF4)**. Casos: **CP-VENTA-001** (venta correcta, descuento stock 20 → 18), **CP-VENTA-002** (venta con descuento), **CP-VENTA-003** (stock insuficiente → 422), **CP-VENTA-004** (la venta guarda el costo del momento). Resultado: **4 / 4 aprobados**.

5. Resumen de la ejecución

Grupo	Archivo	Casos	Cobre	Resultado
Productos (requisitos)	<code>productos.test.js</code>	11	RF1, RF2, RF3, RF4, RF6, RF7, RF8	11 / 11
Ganancias (requisitos)	<code>ganancias.test.js</code>	4	RF5, RF7	4 / 4
Autenticación (adicional)	<code>auth.test.js</code>	5	soporta RF8	5 / 5
Ventas / POS (adicional)	<code>ventas.test.js</code>	4	refuerza RF4	4 / 4
Total	—	24	RF1–RF8 (8/8)	24 / 24 (100 %)

Los 8 requisitos funcionales quedan cubiertos y aprobados. La ejecución completa se reproduce con `docker compose up -d y cd pruebas && npm test`.